

Deep Research and Iteration Report

Table of Contents

RM 简易步兵电控考核 v2 深度调研与迭代报告

结论

v2 的目标不是把代码堆得更复杂，而是把 v1 从“作业参考包”升级成“更接近 RoboMaster 电控工程习惯的学习包”。核心迭代有三点：

1. 新增 `safety.c`：把急停、遥控丢失、Pitch 越界、输出限幅等安全状态集中管理。
2. 新增 `c620_can.c`：补上 RoboMaster 常见 C620/M3508 CAN 电调协议打包和反馈解析。
3. 新增调研报告和答辩口径：让提交者能解释设计取舍，而不是只会说“代码能跑”。

调研范围

本轮调研聚焦“如何让 STM32 HAL 简易步兵电控作业更像真实 RM 电控工程”。

优先级：

- 官方硬件资料：RoboMaster C 型开发板、C620 电调手册。
- 官方代码/示例资料：STMicroelectronics STM32Cube HAL 示例、RoboMaster 开发板示例仓库。
- 已有作业 PDF：本地抽取的考核要求。

关键发现

1. C 型开发板决定了硬件适配层应该集中隔离

RoboMaster C 型开发板面向机器人控制，典型接口包含 STM32 主控、IMU、CAN、UART、PWM、DBUS 等。对考核作业来说，最重要的启发不是照搬某块板子的初始化代码，而是要把硬件接口隔离出来。

因此 v2 继续保留 `bsp_port.c`：

- 控制模块不直接调用 HAL。
- 换开发板时只改 BSP 层。
- 文档里明确说明哪些是真实硬件待接入。

2. C620/M3508 CAN 协议是 RM 电控常见能力，应作为加分理解点

虽然题目写的是普通直流霍尔编码器电机，但 RM 场景里常见的是 C620 电调配 M3508 电机。C620 手册给出了标准 CAN 控制和反馈格式，因此 v2 新增独立 `c620_can` 模块：

- `C620_PackCurrentFrame()`：把 4 路电流命令打包成 8 字节 CAN 帧。
- `C620_ParseFeedback()`：解析 0x201-0x208 反馈帧，得到角度、转速、转矩电流、温度。
- `C620_LimitCurrent()`：把电流限制在合法范围。

这个模块不强制用于题目基本要求，但能在 README 或答辩中作为“我了解 RM 常见电机通信方式”的加分点。

3. 安全保护应该独立成状态机，而不是散落在 if 判断里

题目要求至少实现输出限幅、Pitch 限位、底盘急停中的两个。v1 已覆盖这些点，但逻辑分散在 `app/chassis/gimbal` 中。v2 把安全判断集中到 `safety.c`：

- `Safety_Update()` 统一更新安全状态。
- `Safety_ShouldStop()` 给调度层一个清晰决策。
- `Safety_ReportMotorLimited()` 让底盘/云台输出限幅可以被记录。

这样的好处是后续可以继续加入：

- 遥控器超时。
- 电机堵转。
- 电调过温。
- IMU 离线。
- 低电压保护。

4. 对考核最有价值的不是“全做完”，而是边界诚实

作业 PDF 明确写了“做多少算多少不要求全部做完”，并强调自主学习解决困难。提交时应该诚实区分：

- 已完成：软件架构、任务调度、运动学、PID 接口、安全保护。
- 已预留：编码器速度、IMU 角度、CAN 电机通信。
- 待实测：具体开发板引脚、电机驱动方向、PID 参数。

这比虚假承诺“已实车验证全部功能”更可信。

v1 到 v2 的具体迭代

代码迭代

- 新增 Core/Inc/safety.h
- 新增 Core/Src/safety.c
- 新增 Core/Inc/c620_can.h
- 新增 Core/Src/c620_can.c
- app.c 接入 Safety_Init()、Safety_Update()、Safety_ShouldStop()
- chassis.c 在输出限幅时上报安全状态
- README 更新 v2 新增模块

文档迭代

- 新增本报告：解释调研依据和迭代原因。
- 保留逐步教程：让提交者按模块真正写懂。

- 保留创作心得模板：但要求必须替换成真实调试经历。

提交者答辩口径

可以这样说：

我先按题目要求实现了模块化、底盘运动学、云台目标角和安全保护。后来我查了 RM 常见硬件，发现真实电控工程里 CAN 电调和集中安全状态很重要，所以 v2 里把 C620 CAN 帧打包和反馈解析单独做成模块，同时把安全判断从控制流程里拆成了 `safety.c`。

我没有把 HAL 代码写进底盘和云台模块，因为不同开发板的按键、PWM、CAN、IMU 接法不一样。把硬件适配放进 `bsp_port.c` 后，算法层就不会被具体引脚绑死。

这份代码目前是应用层框架和可移植参考，真实电机、编码器和 IMU 需要接入板子后继续验证。我会在测试记录里明确区分已软件验证和待实车验证的部分。

调研来源

- RoboMaster C 型开发板产品页：
<https://www.robomaster.com/en-US/products/components/general/development-board-type-c>
- RoboMaster C620 Brushless DC Motor Speed Controller 用户手册：<https://cdn-hz.robomaster.com/robomasters/public/document/RoboMaster%20C620%20Brushless%20DC%20Motor%20Speed%20Controller%20V1.0.pdf>
- STMicroelectronics STM32CubeF4 官方 HAL 示例仓库：
<https://github.com/STMicroelectronics/STM32CubeF4>
- RoboMaster Development Board C
Examples: <https://github.com/RoboMaster/Development-Board-C-Examples>
- 本地作业 PDF 抽取文本：
`C:/Temp/codex_rm_assignment/rm_control_assignment_extracted.md`

边界声明

本报告没有声称 v2 已经在真实机器人上跑通。当前已完成的是软件结构、协议打包/解析和本地 C 语法检查；真实硬件验证必须在 STM32Cube 工程中完成。